



Article

# Token optimization: The backbone of effective prompt engineering

Advancing prompt engineering through strategic token optimization

By Supal Kanti Chowdhury



2



On this page

## Introduction

In the rapidly advancing domain of generative AI (gen AI), the precision and efficiency of large language models (LLMs) are of utmost importance. Prompt engineering and token optimization are two pivotal elements that substantially impact the performance and cost-effectiveness of these models. This article aims to deliver a comprehensive understanding of these concepts, coupled with practical strategies and illustrative examples to facilitate the implementation of robust and scalable gen AI solutions.

This article offers an in-depth exploration of prompt engineering and token optimization, emphasizing the critical role of token optimization in the design of effective prompt engineering. It also includes practical examples and best practices to help you in your understanding.

## Overview

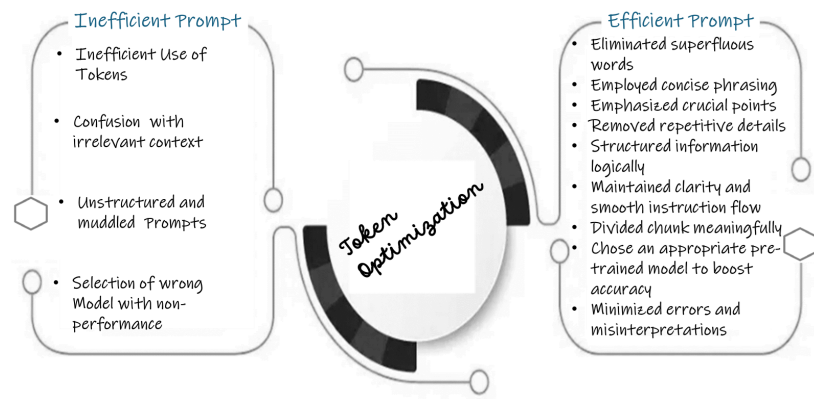
### Introduction

### Understanding prompt engineering

### Understanding tokens in prompt engineering

### Token optimization: a key driving factor for prompt engineering

### Why token optimization matters



# Understanding prompt engineering

First, let's understand what prompt engineering is. It's a method that involves creating input prompts to guide an LLM to generate the desired output. It's similar to giving clear instructions in natural language to a human to ensure that they understand the task.

Prompt engineering is a new and crucial mechanism that helps optimize how you apply, develop, and understand LLMs to generate accurate responses. Technically, it involves designing prompts and interactions to expand the capabilities of language technologies, address their limitations, and gain insights into their functioning.

Effective prompt engineering can significantly improve the accuracy and relevance of LLM responses. By providing the right context and clear instructions, you can guide the model to produce more precise and useful outputs.

There are different components of prompt engineering:

- **Setting the context:** Providing the necessary background information to frame the task
- **Clear instruction:** Ensuring that the instructions are unambiguous, precise, clear, and easy to understand

- Provisioning example (Hints): Including examples and clues to illustrate the desired output format

# Understanding tokens in prompt engineering

In prompt engineering, a *token* is the smallest text unit processed by an LLM, often smaller than a word, such as subwords or characters. Using tokens helps manage out-of-vocabulary words, reduces vocabulary size, and enhances model efficiency. For instance, "unhappiness" might be tokenized into ["un", "##happy", "##ness"] and "playing" be tokenized into ["play", "##ing"]. In both the cases, the "##" symbol indicates that the following token is a subword that should be attached to the previous token.

Every language model has token limits (for example, 2048 or 4096 tokens) for input user tasks or output responses, and exceeding these can lead to incomplete processing. "Context length" refers to the number of tokens that a language model can process at once to generate a response. It's essentially the model's working memory and determines how much information you can provide to the model in a single prompt. For example, a model can generate a response of up to 1548 tokens based on the 500-token prompt where the model context length is 2048 tokens, the prompt length is 500 tokens, and the maximum possible completion length is  $2048 - 500$  (that is, 1548 tokens).

Therefore, counting tokens and context length in prompts are essential to staying within model limits. (There are many tools available for this purpose.) Understanding and considering tokens, which craft more efficient prompts, is crucial for effective communication with language models.

# Token optimization: a key driving factor for prompt engineering

Token optimization is a key driving factor for prompt engineering because it directly impacts the efficiency, cost, and performance of LLMs. Token optimization is crucial for effective prompt engineering because it can:

- **Increase cost efficiency:** Each token processed by an LLM incurs a cost. By minimizing the number of tokens, you can significantly reduce the financial burden of running the model. This is particularly important for commercial applications where cost management is critical. Examples of how it can save costs are:
  - **Before optimization:** Consider a sample prompt: *'Please provide a detailed summary of the customer's purchase history, including all items purchased, dates of purchase, and total amount spent.'* This prompt contains 25 tokens, costs \$0.025 and has a 4-second response time.
  - **After token optimization:** If you optimize the prompt by reducing the tokens with the same intent to *'Summarize the customer's purchase history,'* you reduce the token count to 7. This lowers the cost to \$0.007 and reduces the response generation time to 2 seconds.
- **Improve performance:** Fewer tokens mean less computational load, which can lead to faster response times. This is essential for real-time applications such as chatbots, virtual assistants, and interactive systems.
- **Optimize resources:** Minimizing the token count helps in optimizing the use of computational resources, which is especially important in large-scale applications.

Efficient resource utilization can lead to better scalability and reliability.

- Improve user experiences: Optimized prompts are typically clearer and more concise, making them easier for users to understand and interact with. This enhances the overall user experience and satisfaction.
- Enrich better performance of LLM: By minimizing the token count, you can focus the model's attention on the most relevant information, leading to more accurate and relevant responses.
- Foster scalability: Token optimization is essential for scaling LLM-based solutions. As the number of users and interactions grows, efficient token usage ensures that the system remains performant and cost-effective.
- Provide competitive advantage: Companies that effectively optimize token usage can offer more cost-effective solutions, gaining a competitive advantage in the market.

## Why token optimization matters

Tokens are the basic units of text that LLMs process while generating responses of user tasks. They can be words, subwords, or even characters, depending on the model being used. Token optimization focuses on minimizing the number of tokens used in a prompt without changing its natural meaning or intent to reduce costs and improve performance.

Token usage directly impacts the cost of running LLMs. Each token processed (input or output) incurs a cost, making it essential to optimize token usage to manage expenses effectively. Efficient token usage can enhance the model's performance by reducing the computational load and improving response times. Token usage while

prompt engineering with clear and concise instructions is what really matters.

The following example shows the accuracy and satisfactory response with an optimized token:

**Before:** *"Please generate a summary of the following text."*

**After:** *"Summarize the text below in 50 words or less."*

You can continuously refine prompts based on the model's output and user feedback to improve accuracy over time.

The following sections look at the token-based, vendor-specific performance.

## IBM watsonx

IBM watsonx LLMs:

- IBM watsonx.ai uses a word-based or subword-based tokenization method for its Granite foundation models.
- The Granite foundation model's tokenization process involves a combination of character-level and subword-level splitting, similar to other LLMs:
  - When you provide text for the Granite model, it first goes through a tokenizer, which splits the text into individual tokens. This involves splitting on spaces (usually), breaking words into subwords, and handling punctuation.
  - Each token then gets mapped to a unique identifier using a predefined vocabulary, a large list of words and subwords that the model understands.
- The IBM watsonx.ai Granite model cost structure is based on the number of tokens processed, typically

with a context length of 8192 (input/output) along with \$0.60/1M tokens.

- There are multiple IBM watsonx.ai Granite models (granite-20b-multilingual, granite-7b-lab, and granite-13b-chat) with a pricing model that is designed to be competitive with other commercial LLMs. For example:
  - Input: *"Analyse the sentiment of the following text: 'I love this product!'"*
  - Tokens: ["Analyse", " the", " sentiment", " of", " the", " following", " text", ":", " 'I", " love", " this", " product", "!'"]
  - Cost: Assuming a cost per token of \$0.0000006, the total cost would be \$0.0000078.

Take a look at [foundation models in watsonx.ai](#).

## OpenAI LLMs

OpenAI LLMs:

- These LLMs use a tokenization method that is based on Byte Pair Encoding (BPE), which breaks down words into subwords. This approach helps in handling out-of-vocabulary words and reduces the overall number of unique tokens.
- The charges are based on the number of tokens processed. Costs are typically broken down into input tokens (tokens in the prompt) and output tokens (tokens in the generated response). For example:
  - Input: *"Please summarize the following text: 'The quick brown fox jumps over the lazy dog.'"*
  - Tokens: ["Please", " summarize", " the", " following", " text", ":", " 'The", " quick", " brown", " fox", " jumps", " over", " the", " lazy", " dog", "']
  - Cost: If the cost per token is \$0.0001, the total cost for this input would be \$0.0017.

See this link for more information on [OpenAI pricing](#).

# Facebook (Meta) Llama LLMs

## Meta LLMs:

- These LLMs use a tokenization method that is similar to OpenAI's, but with some differences in the vocabulary and token splitting rules.
- They are designed to handle a wide range of languages and scripts efficiently (Llama models).
- The cost structure for Llama models is not as straightforward as OpenAI's because Meta's models are often used in research settings rather than commercial APIs.
- The principles of token optimization still apply to reduce computational load and improve performance. For example:
  - Input: *"Translate the following text to French: 'Hello, how are you?'"*
  - Tokens: ["Translate", " the", " following", " text", " to", " French", ":", " 'Hello", ",", " how", " are", " you", " "?""]
  - Cost: Assuming a similar cost per token, the total cost would be \$0.0013.

See this link for more information on [Meta LLMs](#).

# Mistral LLMs

## Mistral LLMs:

- These LLMs use a tokenization method that is optimized for multilingual support and efficient processing of various scripts. The tokenization process involves a combination of character-level and subword-level splitting.
- The cost structure is similar to other commercial LLMs, with costs based on the number of tokens processed. The focus is on optimizing token usage to reduce costs and improve performance. For example:

- Input: *"Generate a summary of the following text: 'Artificial intelligence is transforming industries.'"*
- Tokens: ["Generate", " a", " summary", " of", " the", " following", " text", ":", " 'Artificial", " intelligence", " is", " transforming", " industries", ""]
- Cost: Assuming a cost per token of \$0.0001, the total cost would be \$0.0016.

See this link for more information on [Mistral pricing](#).

## Anthropic LLMs

Anthropic LLMs:

- These LLMs use a tokenization method that is designed to handle a wide range of languages and scripts efficiently. The tokenization process involves a combination of character-level and subword-level splitting, similar to other LLMs.
- The cost structure is based on the number of tokens processed, with a focus on optimizing token usage to reduce costs.
- The pricing model is designed to be competitive with other commercial LLMs. For example:
  - Input: *"Analyse the sentiment of the following text: 'I love this product!'"*
  - Tokens: ["Analyse", " the", " sentiment", " of", " the", " following", " text", ":", " 'I", " love", " this", " product", ""]
  - Cost: Assuming a cost per token of \$0.0001, the total cost would be \$0.0014.

See this link for more information on [Anthropic pricing](#).

## Comparative analysis

Comparing all of the models, you can see:

- IBM watsonx LLM: IBM watsonx Granite is highly efficient in terms of tokenization, cost, multilingual support, and processing due to its optimized architecture, efficient inference, and training on diverse data sets. It offers gen AI architecture patterns like Q&A, summarization, classification, generation, extraction, translation, and RAG tasks in French, German, Japanese, Portuguese, Spanish, and English.
- OpenAI: Is efficient for English and other languages with a rich vocabulary. It has a clear cost structure that is based on input and output tokens. It has a high performance with efficient tokenization.
- Meta Llama: Is optimized for multilingual support and efficient processing. The cost structure varies, but token optimization is crucial for research settings. It is optimized for performance in multilingual settings.
- Mistral: Is designed for multilingual support with a focus on character-level and subword-level splitting. It has a competitive cost structure with a focus on token optimization. It has a high performance with efficient tokenization for various scripts.
- Anthropic: Is efficient for a wide range of languages and scripts. It has a competitive pricing model that is based on token usage. It has a high performance with a focus on efficient token usage.

## Techniques for token optimization

There are multiple techniques that you can follow for token optimization.

## Minimizing token count

Minimizing the token count is a strategy that is aimed at reducing the number of tokens used in prompts and responses to improve the efficiency and cost-effectiveness of LLMs. Tokens are the basic units of text that LLMs process, and each token incurs a cost. By minimizing the token count, you can lower the overall cost of running the model and enhance its performance by reducing computational load.

The following are some key considerations by which you can minimize the token count:

- Craft prompts that are clear with concise instructions and to the point. Avoid unnecessary details and repetitions.
- Use abbreviations and acronyms where appropriate and which are publicly acceptable and popular to reduce the number of tokens. For example, use NASA instead of 'National Aeronautics and Space Administration' or use INR instead of Indian Rupees.
- Remove redundant, duplicated words or unnecessary information from the prompt.
- Use models that are optimized for token efficiency, such as those with subword tokenization. For example:

**Before:** *"Please provide a detailed analysis of the company's financial performance."*

**After:** *"Analyze the company's financial performance."*

**Before:** *"Translate the following English text to French: 'Hello, how are you?'"*

**After:** *"Translate to French: 'Hello, how are you?'"*

**Before:** *"The robot is named R2D2 and lives in a futuristic city. It is designed to assist humans in various tasks."*

**After:** *"R2D2 is a robot in a futuristic city designed to assist humans."*

The following are some key aspects of minimizing token count:

- Each token processed by an LLM has an associated cost. Reducing the number of tokens directly lowers the financial cost of using the model.
- Fewer tokens mean less computational load, which can lead to faster response times and more efficient processing.
- Minimizing the token count helps in optimizing the use of computational resources, which is especially important in large-scale applications.

## Efficient context provision

Efficient context provision is the practice of supplying the necessary background information to an LLM in a concise and effective manner. The goal is to provide enough context for the model to generate accurate and relevant responses without using an excessive number of tokens. This approach ensures that the model understands the task at hand while optimizing for cost and performance.

The following are some key considerations by which you can efficiently provision the context:

- Organize contextual information into bullet points, lists, or right alignments of paragraphs to make it easier for the model to process.
- Remove redundant or unnecessary information from the context, such as repeating the same word or meaning.
- Highlight the most important details that are directly relevant to the background of the task.
- Provide context in a structured format that is easy for the model to understand. For example:

**Before:** *"The company is a leading manufacturer of electronic devices. It has been in business for over 50 years and has a strong market presence."*

**After:** *"The company is a leading electronic device manufacturer with a 50-year history and strong market presence."*

**Before:** *"The company is a leading manufacturer of electronic devices. It has been in business for over 50 years and has a strong market presence. The company is known for its innovative products and excellent customer service. Please summarize the company's history and achievements."*

**After:** *"Company Profile:*

*Leading manufacturer of electronic devices*

*In business for over 50 years*

*Strong market presence*

*Known for innovative products and excellent customer service Summarize the company's history and achievements."*

The following are some key aspects of efficient context provision:

- The context should be clear and unambiguous, ensuring that the model understands the task and the relevant background information.
- The context should be provided in a concise manner, avoiding unnecessary details and repetitions to minimize token usage.
- The context should be directly relevant to the task, focusing on the most important information that the model needs to generate an accurate response.
- The context should be structured in a way that is easy for the model to parse and understand. This can include using bullet points, lists, or other formatting techniques.

# Effective chunking

By strategically dividing information into smaller parts, prompt engineers can help the model concentrate on the most pertinent sections of the text, thus reducing potential errors and misinterpretations. This chunking technique allows you to do prompt engineering with more organized and clear prompts, ultimately improving the language model's overall performance and generate accuracy.

You can save and optimize tokens by chunking the complex information semantically, not just cutting sections of paragraphs from the complex text. Chunking semantically involves breaking down a text or prompt into smaller, meaningful units or chunks that maintain the overall semantic coherence. This can help in optimizing the use of tokens and improving the efficiency of language model interactions. Here's how you can apply semantic chunking to the given text:

**Before optimization:** Consider a sample prompt, for example, *"Please provide a detailed summary of the customer's purchase history, including all items purchased, dates of purchase, and total amounts spent."* which consists of 25 tokens, costs \$0.025, and has a 4-second response time.

**After token optimization with semantic chunking:** Chunk 1: *"Summarize the customer's purchase history."* [7 Tokens], Chunk 2: *"Include items purchased, dates, and total amounts spent."* [10 tokens]. That reduces to 17 tokens, costs \$0.017, and has a 3-second response time.

By breaking the prompt into semantically coherent chunks, you maintain the clarity and intent of the original request while reducing the token count, cost, and response time. This approach ensures that the model still receives all necessary information but in a more concise and efficient manner.

# Leveraging pretrained models

Utilize models that are optimized for token efficiency, such as those with subword tokenization.

Using pretrained models is a strategy that involves utilizing existing models that have been trained on large data sets to perform specific tasks. These models have already learned a wide range of patterns and structures from the data, making them highly effective for various applications. By using pretrained models, you can save time and resources that would otherwise be spent on training a model from scratch. Additionally, pretrained models often come with optimized tokenization methods, which can further enhance the efficiency and performance of your applications.

Following are some key considerations by which you can leverage pretrained models:

- Choose a pretrained model that is well-suited to your specific task. Different models are optimized for different types of tasks, such as text generation, translation, and summarization.
- Fine-tune the pretrained model on your specific data set to adapt it to your particular use case. This involves further training the model on a smaller data set that is relevant to your task.
- Take advantage of the optimized tokenization methods that come with pretrained models to minimize token usage and improve performance.
- Integrate pretrained models into your existing systems to enhance their capabilities without the need for extensive redevelopment.
- Continuously update and retrain the pretrained model as new data becomes available to ensure that it remains effective and relevant.

For example:

## Text Summarization

Task: Summarize a news article.

Pretrained model: T5 (Text-to-Text Transfer Transformer)

Fine-tuning: Fine-tune T5 on a data set of news articles and their summaries.

Optimized tokenization: T5 uses SentencePiece tokenization, which is efficient for handling a wide range of languages and scripts.\*

Following are some key aspects of using pretrained models:

- Pretrained models have already undergone extensive training, saving you the time and computational resources required for training a model from scratch.
- Pretrained models are typically trained on large and diverse data sets, making them highly effective for a wide range of tasks.
- Many pretrained models come with optimized tokenization methods, which can help minimize token usage and improve performance.
- Pretrained models can be fine-tuned on specific tasks, allowing you to use their existing knowledge while adapting them to your specific needs.
- Pretrained models are often designed to scale efficiently, making them suitable for large-scale applications.

## Natural language

Token optimization can indeed be achieved through natural language techniques. The following are some ways these techniques can help optimize token usage:

- Sentiment and polite phrases words expressing sentiment or politeness (for example, please, kindly, and appreciate) can often be removed or simplified to reduce token count without altering the core intent of the prompt. AI models typically focus on understanding the main objective rather than the tone. For example, "Could you please help me find some interesting books on AI? This will really be appreciable." can be simplified to "Recommend some interesting books on AI."
- Using clear, emphasized, or focused words direct language can reduce token count by making your prompt more concise. Instead of vague or roundabout phrases, be straightforward. For example, instead of saying "I'm looking for some information about the capital of France, if you could provide that it would be great," you can simply say "What is the capital of France?" However, words like always, only with, and ensure should not be removed because they can change the meaning by adding conditions or emphasis. For instance, "Always starting with 'Thanks' when responding with no additional text" is different from "Respond start with 'Thanks'" because the former implies no additional text should ever be added.
- Filler words such as just, actually, basically, you know, or like can often be removed without changing the meaning of your prompt. These words are used in natural language to smooth speech flow but don't add much meaningful information. For example, "I just need to know the basic steps to bake a cake, actually" can be simplified to "What are the steps to bake a cake?"
- Avoiding repetition ensure that you're not repeating information unnecessarily. AI models can typically understand your intent without needing repetition for emphasis. For example, instead of saying "I need to know the weather for today, the current day's weather forecast," you can simply say "What is today's weather forecast?"

By using these natural language techniques, you can make your prompts more concise and optimize token

usage while still clearly conveying your intent to the AI model. It's about finding the right balance between conciseness and clarity.

# Three prompt techniques and their respective token optimization strategies

There are various strategies for optimizing tokens for different prompts. The following strategies are for optimizing tokens in zero-shot, few-shot, and chain-of-thought prompting.

## 1. Zero-shot prompting

Zero-shot prompting is where you provide a task to generate a response without giving the model any further example. The model is given one instance of an input-output pairing to understand the ask and generate a response based on that single example.

## Token optimization strategies

The token optimization strategies for zero-shot prompting are:

- Use short, clear examples that convey the necessary information without unnecessary details.
- Focus on the core elements of the example that are crucial for the model to understand the task.
- Remove any repetitive or redundant information.

- Use a structured format (for example, bullet points or tables) to present the example efficiently.

An example of this is:

Input: *"Translate 'Hello' to Hindi."*

## 2. Few-shot prompting

Few-shot prompting provide multiple examples to help the model better understand the ask. In this type of prompt, you provide more than one example of both the task and the output that you want.

## Token optimization strategies

The token optimization strategies for few-shot prompting are:

- Choose a few highly representative examples that cover the range of possible inputs and outputs.
- Keep the context around each example to a minimum.
- Use a consistent format for all examples to reduce the need for additional explanatory text.
- Batch similar examples together to reduce the overhead of repeated instructions (if possible).

## 3. Chain-of-thought prompting

Chain-of-thought prompting guides the model through a series of logical steps to arrive at a final answer. In this type of prompt, you can use both zero-shot and few-shot prompting techniques along with using the phrase "Think step by step" to invoke reasoning from the model.

# Token optimization strategies

The token optimization strategies for chain-of-thought prompting are:

- Break down the thought process into concise, logical steps.
- Provide clear instructions for each step without unnecessary elaboration.
- Where appropriate, use abbreviations or shorthand to save tokens.
- Include only the most relevant intermediate results to keep the chain concise.

## Best practices for implementation

Some best practices for implementation are:

- Document all prompts and their performance metrics to facilitate continuous improvement.
- Regularly test and validate prompts by using a diverse set of inputs to ensure robustness.
- Establish a feedback loop to iteratively refine prompts based on user feedback and model performance.

## Conclusion

This article provided guidelines and examples to give you a comprehensive understanding of the critical roles that prompt engineering and token optimization play in enhancing the accuracy, efficiency, and cost-effectiveness of generative AI solutions. By meticulously crafting prompts and optimizing token usage, the performance and scalability of AI models can be improved significantly.

The strategies and best practices outlined in this article provide a robust framework for designing and implementing advanced AI solutions that are not only effective but also economical.

By adhering to these guidelines, you can see how generative AI models can be optimized for both performance and resource utilization, thereby maximizing their potential in various applications. Whether you're developing natural language processing systems, generating creative content, or automating complex tasks, the principles of prompt engineering and token optimization are essential for achieving state-of-the-art results.

In summary, the integration of these techniques is fundamental for building robust, scalable, and cost-effective gen AI solutions that can meet the demands of today's rapidly evolving technological landscape.

## References

- [LLM provider documentation](#)
- [Prompt engineering playground](#)
- [Token Analyzer](#)
- [Advanced prompt engineering techniques](#)
- [Token optimization strategies](#)

15 September 2024

Time to read: 23 minutes

Legend ⓘ

---

### Categories



[Generative AI](#)   [Artificial intelligence](#)

[Large language models \(LLMs\)](#)   [Granite models](#)

**Related Topics**



[Prompt Engineering Fundamentals](#) →

**Interested in generative AI?**

[Learn generative AI skills](#) →



**IBM Developer**

- [About](#)
- [FAQ](#)
- [Third-party notice](#)

**Follow Us**

- [X](#)
- [LinkedIn](#)
- [YouTube](#)

**Explore**

- [Open Source @ IBM](#)
- [IBM API Hub](#)

[Career Opportunities](#)

[Privacy](#)

[Terms of use](#)

[Accessibility](#)

[Cookie preferences](#)

[Sitemap](#)